

Introduction to Programming in C

UNIT II: Control Flow and Branching

BY: RISHI KUMAR

Conditional Statements

- Conditional statements are used to execute a set of statements on some conditions.
- In C program, statements are executed sequentially, if no condition is given around the statements.
- Flow of statement execution can be changed using conditional statements.
- The process of using conditional statement is known as decision making in 'C.'
- Also called as control statements.

If Statement

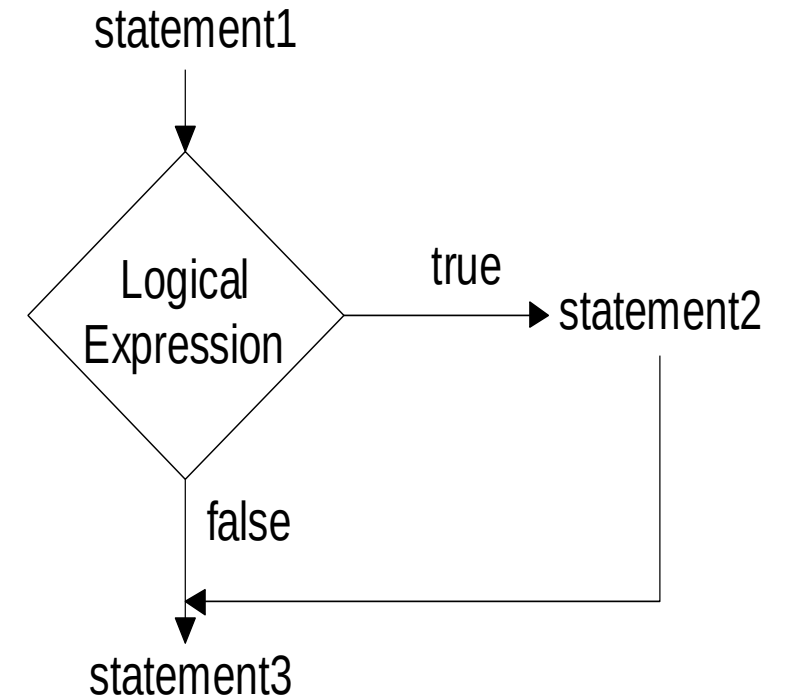
- If the expression is true, then if block statement(s) are executed.
- If the expression is evaluated to false, then control passes to the next statement following it.
- **Syntax:** *if (LogicalExpr) Statement*
- **Program context:**

statement1;

if (LogicalExpr) statement2;

statement3;

Flow of Execution



If statement

Example:

```
void main()
{
    int a;
    printf("\n enter any number");
    scanf("%d", & a);
    if(a%2==0)
    {
        printf("even");
    }
    getch();
}
```

If-else

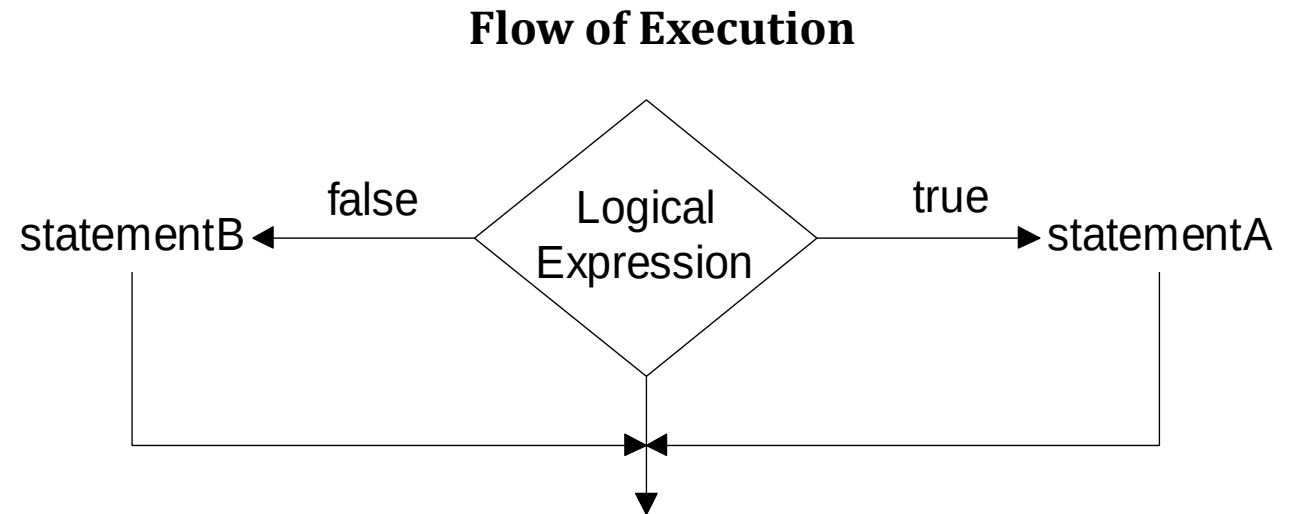
- Used to execute the code if condition is true or false. It is also called two-way selection statement.

- **Syntax:**
if (*LogicalExpr*)

StatementA

else

StatementB



If-else

Example:

```
void main()
{
    int a;
    printf("\n enter any number");
    scanf("%d", & a);
    if(a%2==0)
    {
        printf("even number");
    }
    else
    {
        printf("Odd number");
    }
    getch();
}
```

Nested if-else

- The nested if-else statement allows you to check for multiple test expressions and execute different codes for more than 2 conditions.
- Also known as multi-way selection statement.

Nested if-else

Example:

```
void main()
{
    int a,b,c;
    printf("\n enter any three number");

    scanf("%d %d %d", & a, &b, &c);

    if(a>b)
    {
        if(a>c)

            printf("a is greatest");

        else
            printf("c is greatest");

    }
}
```

```
else
{
    if(b>c)

        printf(" b is greatest");

    else

        printf(" c is greatest");

}
getch();
}
```


If-else-if

- Used to execute one code from multiple conditions.
- It is also called multipath decision statement.
- The if-else-if statement has the following form (3 levels example).

```
if(condition_1)
    statement_1;
else if (condition_2)
    statement_2;
else if(condition_3)
    statement_3;
else
    statement_4;
next_statement;
```

If-else-if

Example:

```
int main()
{
    int marks=80;
    if(marks>75)
    {
        printf("First class");
    }
    else if(marks>65)
    {
        printf("Second class");
    }
```

```
else if(marks>55)
{
    printf("Third class");
}
else
{
    printf("fourth class");
}
return 0;
```

Switch Statement

- A switch statement contains one or more case labels which are tested against the switch expression.
- When the expression match to a case then the associated statements with that case would be executed.
- Acts as a substitute for a long if-else-if ladder.
- Float variables can not be used as switch expression or case values.

Switch Case Syntax

```
switch( expression )
{
    case value-1:
        Block-1;
        Break;

    case value-2:
        Block-2;
        Break;

    case value-n:
        Block-n;
        Break;

    default:
        Block-1;
        Break;
}
Statement-x;
```

Rules for switch statement:

- An expression must always execute to a result.
- Case labels must be constants and unique.
- Case labels must end with a colon (:).
- A break keyword must be present in each case.
- There can be only one default label.
- We can nest multiple switch statements.

Switch Statement format

```
switch (expression)    // expression can not be floating point value.
{
    case value1:        //case value depends on the type of switch expression. Cases
                        //can never have variable expressions like(a+3)

    //Statements

    break;              //used to come out from the switch loop
case
case value 2:
    //Statements
    break;
case value n:
    //Statements
    break;
default:
    //Statements
}
```

Switch Statement

Example:

```
void main ()
{
    int a,b, ch;
    printf("\n enter two numbers")
    scanf("%d%d", &a, &b);
    printf("\n enter choice between
    1,2,3");
    scanf("%d", & ch);

    switch(ch)
    {
        case 1:
            printf("\n sum=%d", a+b);
            break;
        case 2 :
```

```
            printf("\n Subtraction =%d', a-b);
            break;
        case 3 :
            printf("\n Multiplication=%d",
            a*b);
            break;
        default :
            printf("\n Invalid choice" );
    }
    printf("\n end of switch statement");

    getch();
}
```

continue Statement

- continue keyword forces the next iteration to take place immediately, skipping any instructions that may follow it.
- The continue statement can only be used inside a loop (for, do-while and while) and not inside a switch-case selection.
- Unlike the break statement, continue does not force the termination of a loop, it merely transfers control to the next iteration.

continue Statement

Example:

```
int main ()
{

    int a = 10;

    do
    {

        if( a == 12)
        {
            a = a + 1;
            continue;
        }
    }
```

```
printf ("value of a: %d\n", a);
    a++;

} while( a < 14 );

return 0;
```

O/P:
10
11
13

break statement

- **break** statement is used to exit from the current loop.
- It is used to indicate a set of statements is finished (and no more should be executed)
- **Syntax:** break;
- When break statement will be encountered inside any loop, the loop will be immediately terminated, and the control resumes at the next statement following the loop.
- Also used in switch statement to terminate a case.

break Statement

Example:

```
int main () {  
    int a = 10;  
    while( a < 15 ) {  
        printf(“%d\n”, a);  
        a++;  
        if( a > 13)  
        {  
            break;  
        }  
    }  
    return 0;  
}
```

O/P:

10

11

12

13

```
#include <stdio.h>
int main()
{
    int a = 10;
    while( a < 20 )
    {
        printf("%d\n", a);
        a++;
        if( a < 10)
        {
            break;
        }
        else
            a++;
    }
    return 0;
}
```

```

#include <stdio.h>
#include <string.h>

// use #define macro to define the values for UserName and Password
#define UserName "system"
#define Password "admin@123"

int main ()
{
    // declare character type array
    char un[50], pass[50];
    // take UserName and Password from user
    printf ( " Enter the username: " );
    scanf ( " %s", un);
    printf ( " Enter the password: " );
    scanf ( " %s", pass);

    // use if statement and Logical AND operator to validate the condition
    if (strcmp (UserName, un) == 0 && strcmp (Password, pass) == 0)
    {
        printf ( " \n The user's credentials are correct. ");
    }
    else
    {
        printf ( " \n The user's credentials are incorrect. ");
    }
    return 0;
}

```

16. WAP to validate the username and password entered by the user is correct or not using the predefined username and password.

Next Topics:

1. goto and exit()

2. Loops

- 2.1 While loop

- 2.2 Do while loop

- 2.3 For loop

goto statement

- goto statement is used for unconditional jumping.
- It is a jumping statement which can be used to jump from one statement to another inside any function.

label:	goto label;
//statements....	//statements...
goto label;	label:

- goto label: used to tell compiler to jump to the statement marked as a label.
- label: user-defined identifier which indicates the target statement.

exit()

- The **exit()** function is a standard library function.
- It terminates the execution of the program.
- It is necessary to use this function with goto statement, if you do not want to execute the statements written after else statement.

goto and exit ()

Example:

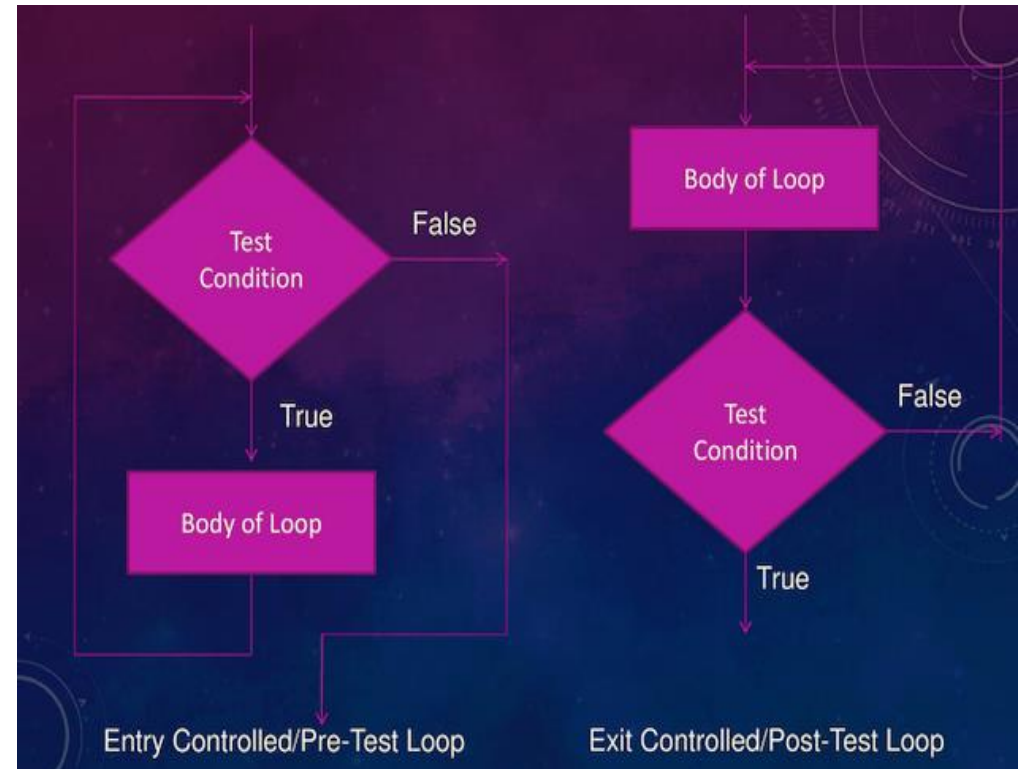
```
#include <stdio.h>
void display()
{
    int n;
    printf("\n enter number");
    scanf("%d", &n);
    if(n<=5)
        goto label;
    else
    {
        printf("\n number is greater than 5");
        exit(0); }
}
```

```
label:
    printf("number is less than or equal to 5");
}

int main()
{
    display();
    return 0;
}
```


Loops

- A loop is a sequence of instructions that is continually repeated until a certain condition is reached.
- Loops allow for the same statement to be executed several times in succession.
- entry-controlled and exit-controlled.
- while, do-while and for loop.
 - for and while loop is entry-controlled loops.
 - do-while is an exit-controlled loop.



The while loop

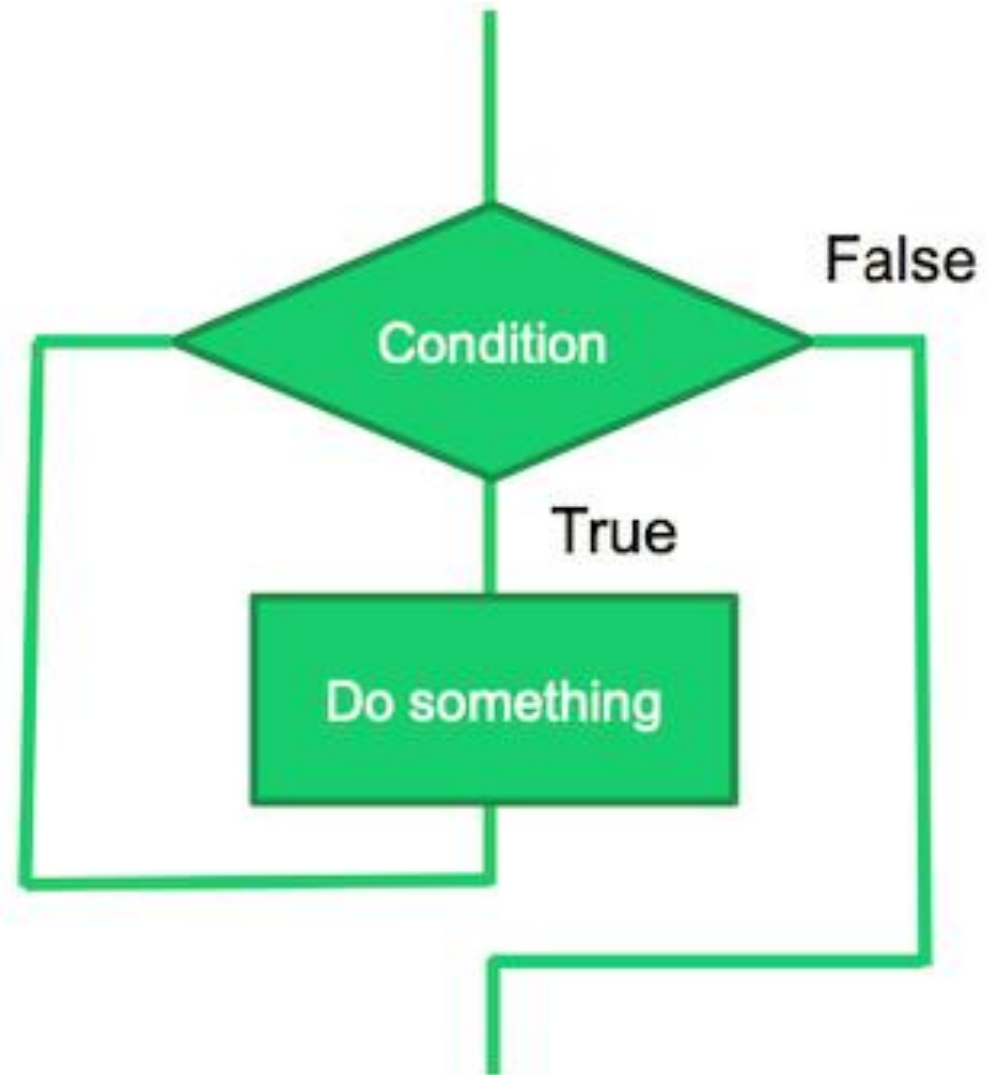
- A WHILE loop is a loop that repeats while some condition is satisfied.
- It tests its condition at the beginning of every loop.
- A ***while loop*** has the following syntax:

while (*condition*)

statement;

- If the condition is true, the statement is executed; then the condition is evaluated again
- If the condition of a **while** statement is false initially, the statement is never executed
- Therefore, we say that a **while** statement executes zero or more times

while loop
- flow
diagram



do-while loop

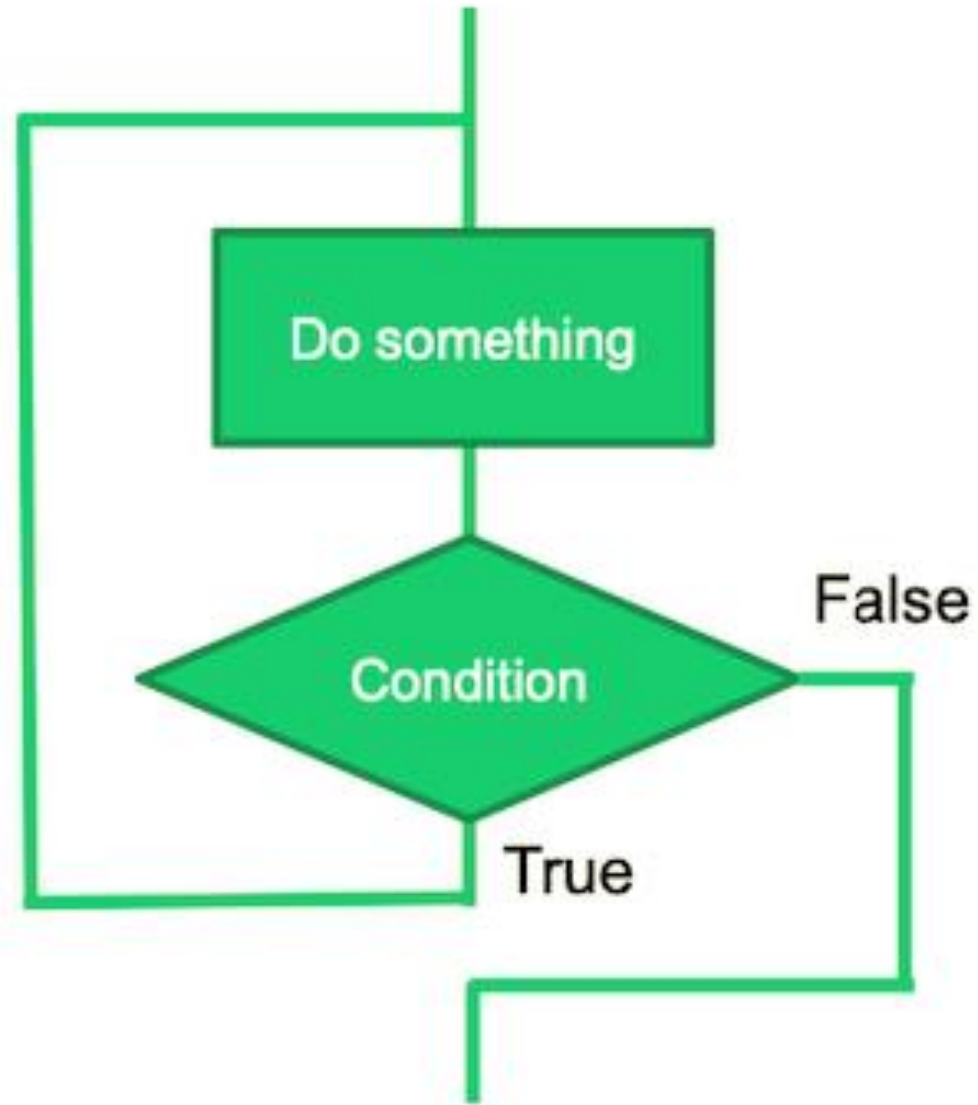
- Unlike a while loop, a do-while loop tests its condition at the end of the loop.
- This means that its sequence of activities always runs at least once.
- Syntax is:

do

statement

while (expression);

do-while
loop - flow
diagram



while and do-while loop

Example:

```
while(condition)
{
    statements;
}
```

Example:

```
int a=5;
while(a<5)
{
    printf("%d\n",a);
}
printf("\n end of loop");
```

O/P: end of loop

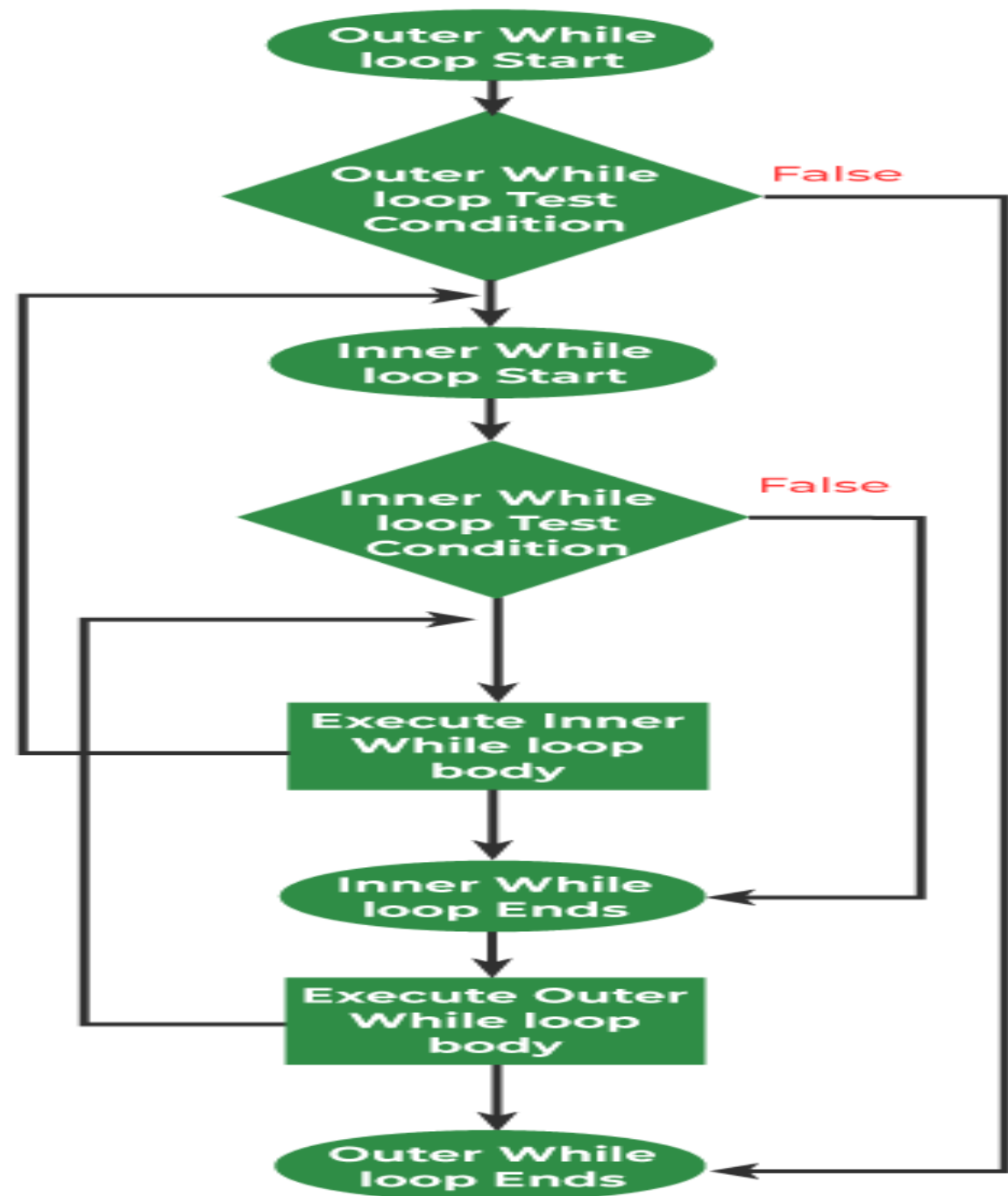
```
do
{
    statements;
} while (expression);
```

Example:

```
int a=5;
do
{
    printf("\n%d", a);
}while(a<5);
printf("\n end of loop");
```

O/P: 5
end of loop

Nested while Loop



Nested while Loop

Syntax:

```
while(condition) {
```

```
    while(condition) {
```

```
        // statement of inside loop
```

```
    }
```

```
    // statement of outer loop
```

```
}
```


Example: Print Pattern using nested while loops

```
// C program to print pattern using nested while loops
#include <stdio.h>
```

```
int main()
{
    int end = 5;

    printf("Pattern Printing using Nested While loop");

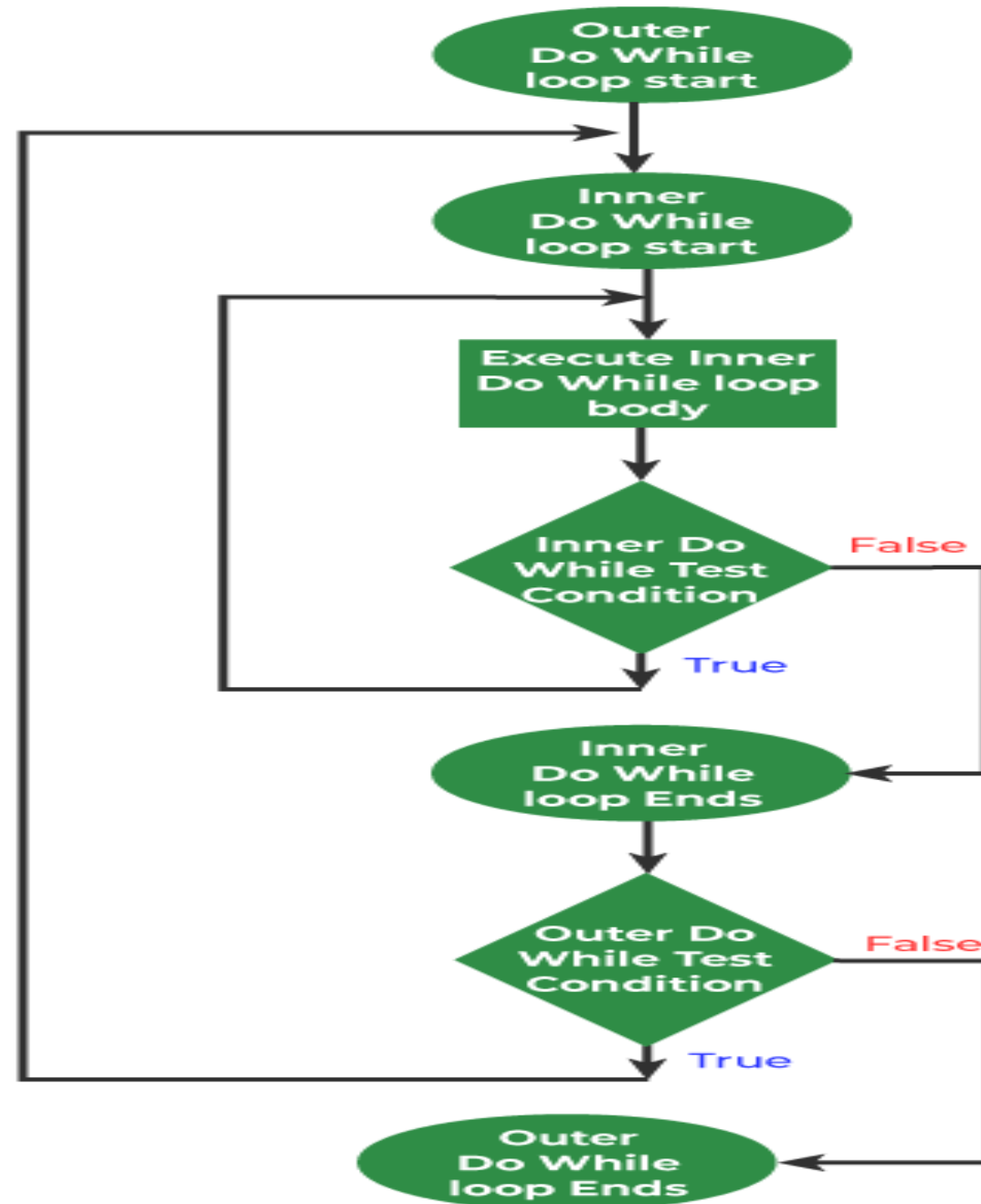
    int i = 1;

    while (i <= end) {
        printf("\n");
        int j = 1;
        while (j <= i) {
            printf("%d ", j);
            j = j + 1;
        }
        i = i + 1;
    }
    return 0;
}
```

Output:

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

Nested do-while Loop



Nested do-while Loop

Syntax:

```
do{
```

```
    do{
```

```
        // statement of inside loop
```

```
    }while(condition);
```

```
    // statement of outer loop
```

```
}while(condition);
```

Program to displays a square number pattern

Square number pattern

Here your pattern

```
12345678910
12345678910
12345678910
12345678910
12345678910
12345678910
12345678910
12345678910
12345678910
12345678910
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i,j;
    i=1;
    printf("Square number pattern\n\n");
    printf("Here your pattern\n\n");
    do{
        j=1;
        do{
            printf("%d",j);
            j++;
        }while(j<=10);
        printf("\n");
        i++;
    }while(i<=10);
    getch();
    return 0;
}
```

program displays a square star pattern

Asterisk number pattern

Here your pattern

```
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****  
*****
```

```
#include <stdio.h>  
#include <stdlib.h>
```

```
int main()  
{  
    int i,j;  
    i=1;  
    printf("Asterisk number pattern\n\n");  
    printf("Here your pattern\n\n");  
    do{  
        j=1;  
        do{  
            printf("*");  
            j++;  
        }while(j<=9);  
        printf("\n");  
        i++;  
    }while(i<=9);  
    getch();  
    return 0;  
}
```

Program displays a floyd's triangle number pattern using nested do-while loop

Triangle number pattern

Here your pattern

```
1
12
123
1234
12345
123456
1234567
12345678
123456789
```

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int i,j;
    i=1;
    printf("Triangle number pattern\n\n");
    printf("Here your pattern\n\n");
    do{
        j=1;
        do{
            printf("%d",j);
            j++;
        }while(j<=i);
        printf("\n");
        i++;
    }while(i<=9);
    getch();
    return 0;
}
```

Print multiplication table using nested do-while loop

```
Multiplication table
Here multiplication table
1    2    3    4    5    6    7    8    9    10
2    4    6    8    10   12   14   16   18   20
3    6    9    12   15   18   21   24   27   30
4    8    12   16   20   24   28   32   36   40
5    10   15   20   25   30   35   40   45   50
6    12   18   24   30   36   42   48   54   60
7    14   21   28   35   42   49   56   63   70
8    16   24   32   40   48   56   64   72   80
9    18   27   36   45   54   63   72   81   90
10   20   30   40   50   60   70   80   90   100
```

```
#include <stdio.h>
#include <stdlib.h>
```

```
int main()
{
    int i,j;
    i=1;
    printf("Multiplication table\n\n");
    printf("Here multiplication table\n\n");
    do{
        j=1;
        do{
            printf("%d\t",i*j);
            j++;
        }while(j<=10);
        printf("\n");
        i++;
    }while(i<=10);
    getch();
    return 0;
}
```

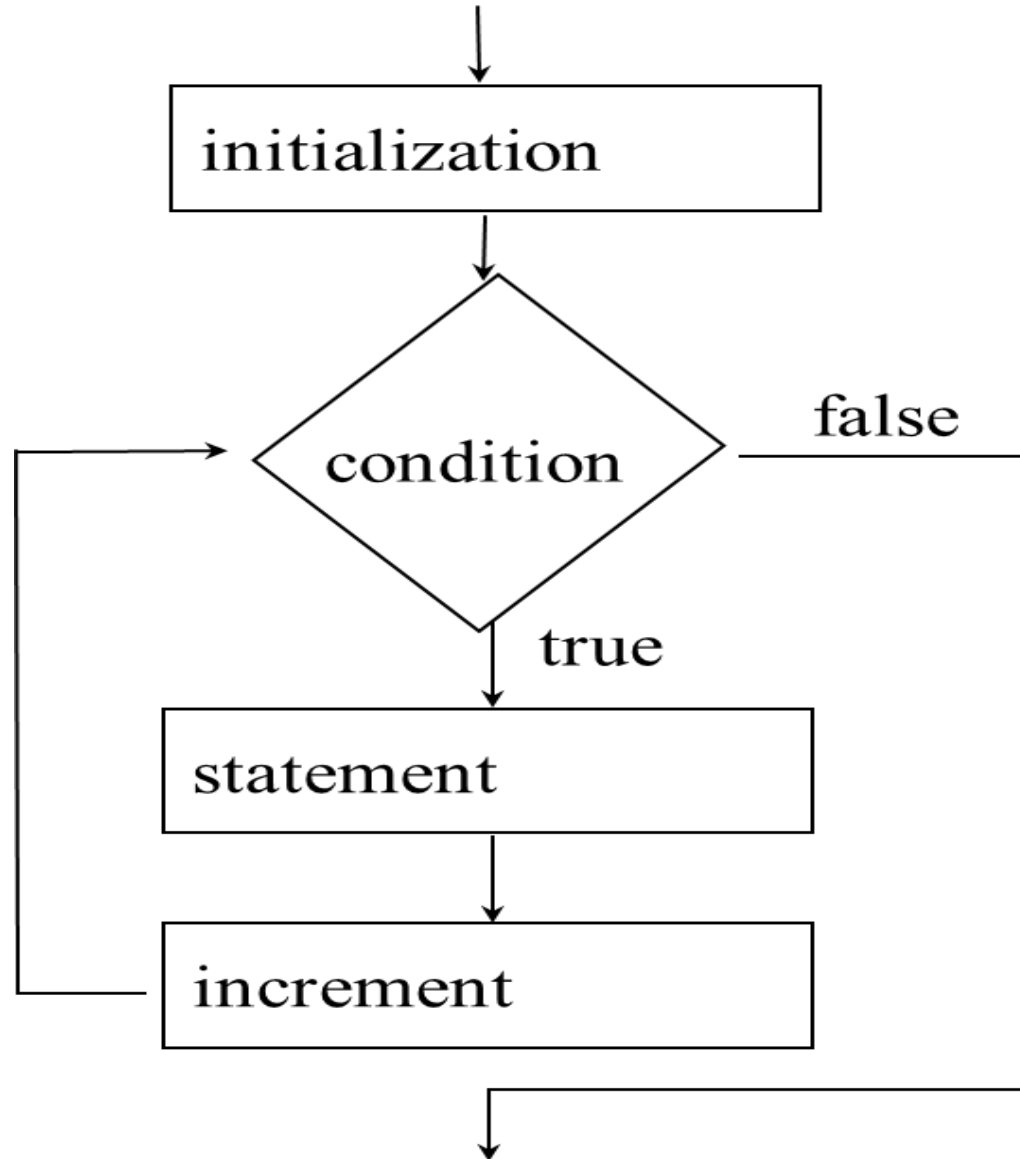
for Loop

- A for loop is a loop that repeats a specified number of times.
- The loop uses a counter to tell it how many times to run the same sequence of activities.

Syntax:

```
for ( init; condition; increment )  
{  
    statement(s);  
}
```


for loop -
Flow
diagram



for loop

Example:

for(i=1;i<=3;i++)

printf("\n %d", i);

O/P: 1

2

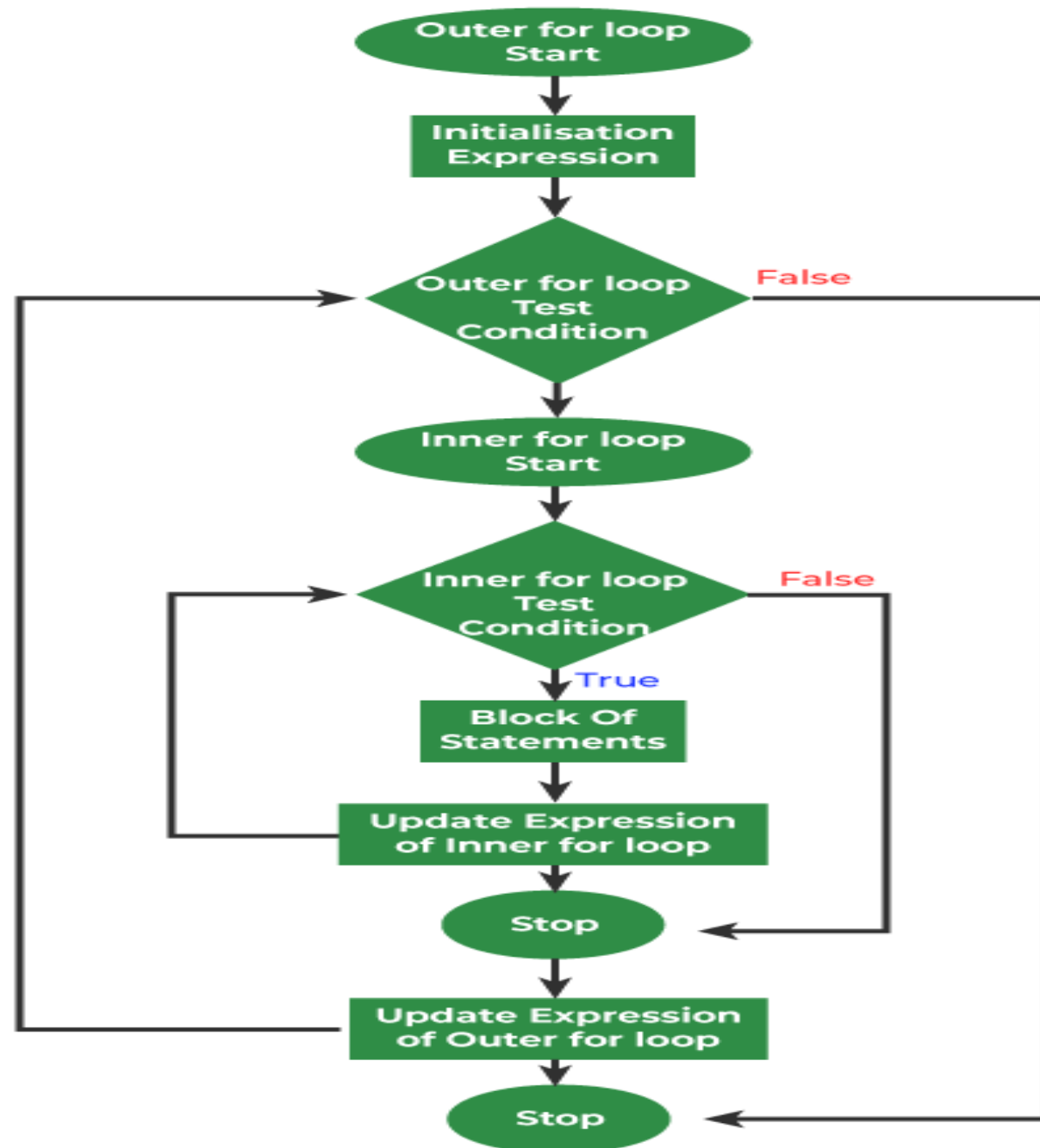
3

NESTED For LOOP

Syntax:

```
for ( initialization; condition; increment ) {  
    for ( initialization; condition; increment ) {  
        // statement of inside loop  
    }  
    // statement of outer loop  
}
```

flow diagram
NESTED For
LOOP



NESTED Multiple LOOPS

Syntax:

```
do{  
  
    while(condition) {  
  
        for ( initialization; condition; increment ) {  
  
            // statement of inside for loop  
        }  
  
        // statement of inside while loop  
    }  
  
    // statement of outer do-while loop  
}while(condition);
```

WAP to Check Whether a Number is Prime or Not

A prime number is a natural number that has only one and itself as factors.

```
#include <stdio.h>
```

```
main() {  
    int n, i, c = 0;  
    printf("Enter any number n:");  
    scanf("%d", &n);  
  
    //logic  
    for (i = 1; i <= n; i++) {  
        if (n % i == 0) {  
            c++;  
        }  
    }  
  
    if (c == 2) {  
        printf("n is a Prime number");  
    }  
    else {  
        printf("n is not a Prime number");  
    }  
    return 0;  
}
```

```
#include <stdio.h>
int main() {
    int n, i, flag = 0;
    printf("Enter a positive integer: ");
    scanf("%d", &n);

    // 0 and 1 are not prime numbers change flag to 1 for non-prime number
    if (n == 0 || n == 1)
        flag = 1;

    for (i = 2; i <= n / 2; ++i) {
        // if n is divisible by i, then n is not prime change flag to 1 for non-prime number
        if (n % i == 0) {
            flag = 1;
            break;
        }
    }

    // flag is 0 for prime numbers
    if (flag == 0)
        printf("%d is a prime number.", n);
    else
        printf("%d is not a prime number.", n);
    return 0;
}
```

// C Program to print all prime factors of a number using nested loops

```
#include <math.h>
#include <stdio.h>
```

```
// A function to print all prime factors of a given number n
void primeFactors(int n)
{
    // Print the number of 2s that divide n
    while (n % 2 == 0) {
        printf("%d ", 2);
        n = n / 2;
    }
```

Output ; 3 3 5 7

```
// n must be odd at this point. So we can skip
// one element (Note i = i + 2)
for (int i = 3; i <= sqrt(n); i = i + 2) {
    // While i divides n, print i and divide n
    while (n % i == 0) {
        printf("%d ", i);
        n = n / i;
    }
}

// This condition is to handle the case when n
// is a prime number greater than 2
if (n > 2)
    printf("%d ", n);
}

/* Driver program to test above function */
int main()
{
    int n = 315;
    primeFactors(n);
    return 0;
}
```


// C program to show working of break statement inside nested for loops

```
#include <stdio.h>
int main()
{
    int i = 0;
    for (int i = 0; i < 5; i++) {
        for (int j = 0; j < 3; j++) {
            // This inner loop will break when i==3
            if (i == 3) {
                break;
            }
            printf("* ");
        }
        printf("\n");
    }
    return 0;
}
```

* * *

* * *

* * *

* * *

// C program to show working of continue statement inside nested for loops

```
#include <stdio.h>
```

```
int main()  
{
```

```
    int i = 0;
```

```
    for (int i = 0; i < 5; i++) {
```

```
        for (int j = 0; j < 3; j++) {
```

```
            // This inner loop will skip when j==2
```

```
            if (j==2) {
```

```
                continue;
```

```
            }
```

```
            printf("%d ",j);
```

```
        }
```

```
        printf("\n");
```

```
    }
```

```
    return 0;
```

```
}
```